Decision and Risk Analysis

Lecture 10: **Bayesian Penguins**

DAEN 427 / ISEN 427 / ISEN 627 - Fall 2026

Alexander Abuabara



Learning goals: the penguin mission

1. Explore a real.
2. Update beliefs using a conjugate Normal model.
3. Fit Bayesian regression in `rstanarm` (and `brms`).
4. Separate *prior*, *posterior*, and *predictive* distributions.
5. Check models, forecast a new penguin, and choose an action.
6. Get started on hierarchical models.

Meet our colleagues



The sample records **Adelie, Chinstrap, and Gentoo** penguins from the Palmer Archipelago. Data: Palmer penguins, Gorman et al. (2014), via `bayesrules/palmerpenguins`.

1. Load packages and the dataset

```r
library(tidyverse)  # dplyr, ggplot2, tidyr, ...
library(bayesrules) # teaching data and conjugate helpers
library(rstanarm)  # Bayesian regression with Stan
library(bayesplot) # Bayesian diagnostic plotting

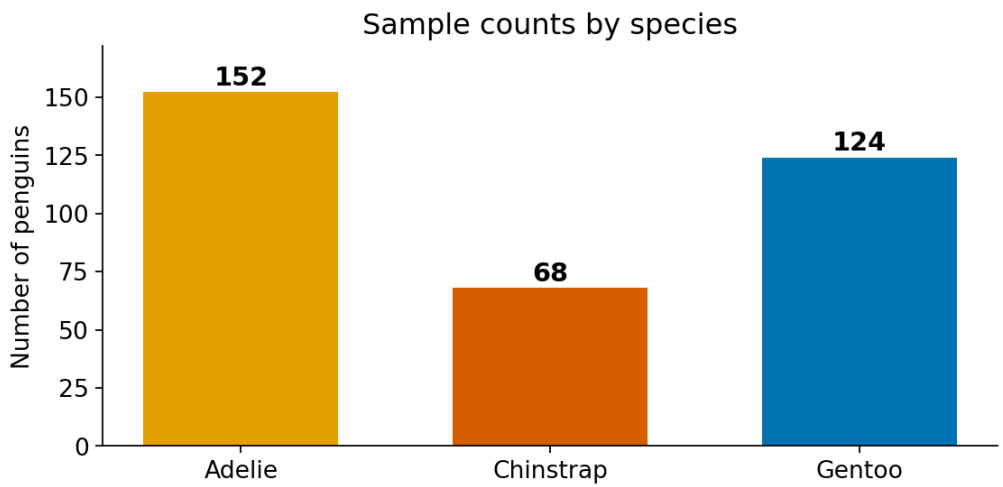
data("penguins_bayes", package = "bayesrules")
penguins <- penguins_bayes
```

- `library()` attaches functions; `data()` loads a packaged object.
- `<-` assigns an R object. `penguins` is a convenient copy/reference.
- 344 rows; 9 columns, including `above_average_weight`.

2. Inspect before modeling

```r
table(penguins$year, penguins$species)
dim(penguins)            # row count and column count
str(penguins)            # base R structure
glimpse(penguins_bayes) # tidyverse structure
penguins %>% count(species)
```

- $ extracts a column; `table()` cross-tabulates two categorical columns.
- %>% pipes an object into the next function.
- `count(species)` groups and counts the three species.



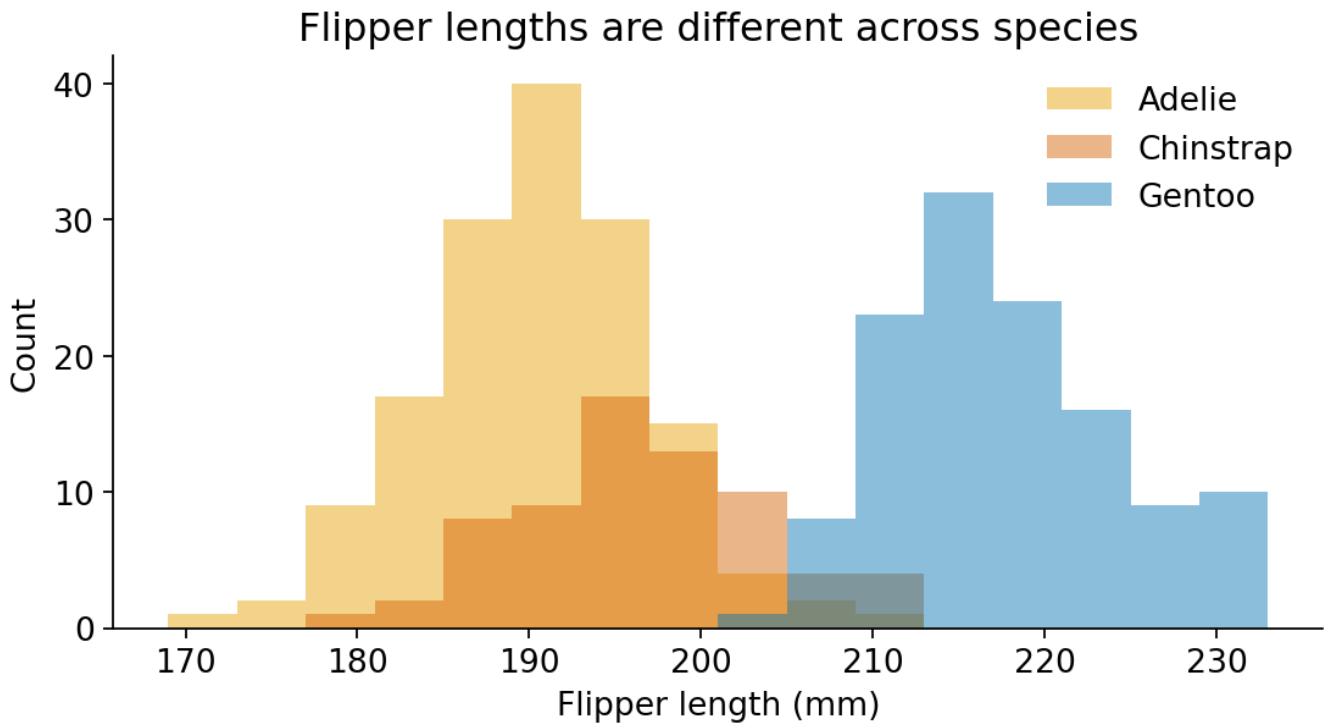
3. Summaries by species

```
penguins %>%
  group_by(species) %>%
  summarise(
    n = n(),
    flipper_mean = mean(flipper_length_mm, na.rm=TRUE),
    bill_mean = mean(bill_length_mm, na.rm=TRUE),
    mass_mean = mean(body_mass_g, na.rm=TRUE),
    .groups = "drop"
  )
```

`group_by()` defines the comparison groups; `summarise()` collapses each group to one row; `n()` counts rows; `na.rm=TRUE` prevents missing observations from making the mean missing; `.groups="drop"` returns an ungrouped result.

One row per species, but each mean uses only nonmissing values in its variable.

Explore flippers: one species or three stories?



If species have different typical flipper lengths, what might a single pooled regression miss?

A good model starts with plots, not just a formula.

Bayes in one sentence

$$\underbrace{p(\theta \mid y)}_{\text{posterior}} \propto \underbrace{p(y \mid \theta)}_{\text{likelihood}} \underbrace{p(\theta)}_{\text{prior}}$$

- θ: unknown parameter; y: observed measurements.
- **Prior**: plausible parameter values before these observations.
- **Likelihood**: which parameter values explain the data.
- **Posterior**: updated parameter uncertainty after observing data.

Bayesian inference produces a distribution, not only a best estimate.

4. Encode a conjugate prior and likelihood

```
mu0 <- 190
prior_sd <- 10
sigma_known <- 7
```

$$\mu \sim N(190, 10^2), \qquad y_i \mid \mu \sim N(\mu, 7^2)$$

- μ: unknown average Adelie flipper length (mm).
- `mu0`: prior mean; `prior_sd`: prior uncertainty.
- `sigma_known`: *assumed known* measurement/population spread.

This known-σ assumption makes the posterior analytically tractable, but should be justified or relaxed in real work.

5. Select observed Adelie measurements

```
y <- na.omit(penguins$flipper_length_mm[
  penguins$species == "Adelie"
])
n <- length(y)
y_bar <- mean(y)
```

Brackets `[condition]` select only matching rows; `na.omit()` excludes missing lengths; `length()` gives sample size; `mean()` computes $\bar{y}$.

```
# Equivalent tidyverse pipeline:
y <- penguins %>% filter(species == "Adelie") %>%
  pull(flipper_length_mm) %>% na.omit() %>% as.numeric()
```

In these records, $n = 151$ nonmissing Adelie flipper lengths, $\bar{y} \approx 189.95$ mm.

6. Derive the Normal–Normal posterior

$$\sigma^2_{\text{post}} = \left(\frac{1}{\tau_0^2} + \frac{n}{\sigma^2} \right)^{-1}$$

$$\mu_{\text{post}} = \sigma^2_{\text{post}} \left(\frac{\mu_0}{\tau_0^2} + \frac{n\bar{y}}{\sigma^2} \right)$$

$$\mu \mid y \sim N(\mu_{\text{post}}, \sigma^2_{\text{post}})$$

Why it works: prior precision $1/\tau_0^2$ and data precision n/σ^2 add. The posterior mean is a precision-weighted compromise between prior center and sample mean.

7. Compute posterior and credible interval

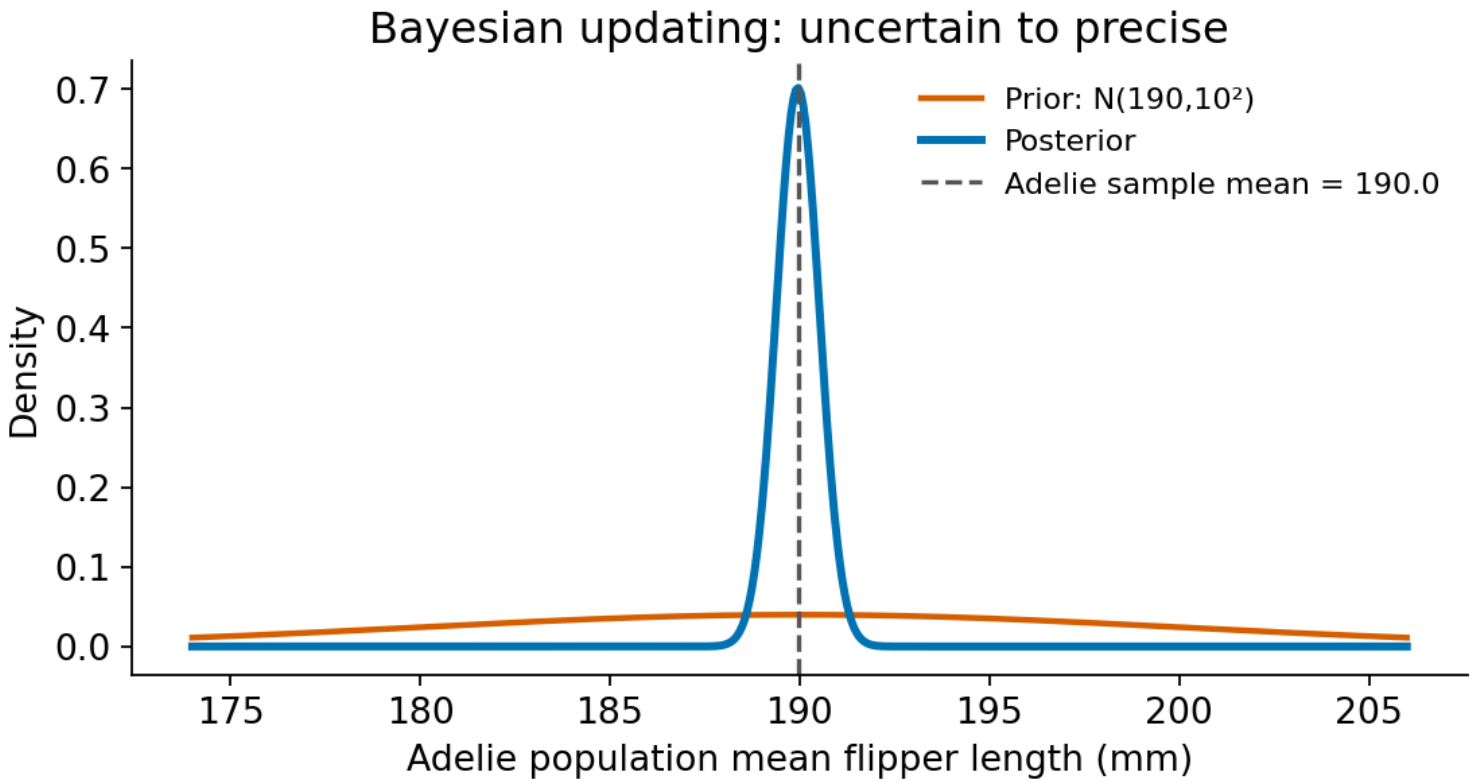
```
post_var <- 1/(1/prior_sd^2 + n/sigma_known^2)
post_mean <- post_var*(mu0/prior_sd^2 +
                         n*y_bar/sigma_known^2)
post_sd <- sqrt(post_var)
credible_interval <- qnorm(c(.025,.975),
                           mean=post_mean, sd=post_sd)
post_mean; post_var; post_sd; credible_interval
```

`qnorm()` returns Gaussian quantiles; `c()` forms a vector; `sqrt()` converts variance to SD.

$$\mu \mid y \approx N(189.954,\ 0.569^2)$$

95% credible interval $\approx [188.84, 191.07]$ mm

What does 95% credible mean?



Interpretation: Conditional on the model, data, and prior, there is 95% posterior probability that μ falls inside the reported interval. This is about the *population mean*, not the flipper length of a future individual.

Remember to know: Why is the posterior much narrower than the prior?

8. Validate and sample from the exact posterior

```
summarize_normal_normal(
 mean=mu0, sd=prior_sd, sigma=sigma_known,
 y_bar=y_bar, n=n
)
post_mu <- rnorm(10000, mean=post_mean, sd=post_sd)
ggplot(data.frame(mu=post_mu), aes(mu)) +
  geom_density(fill="steelblue", alpha=.4) +
  labs(title="Posterior: mean Adelie flipper length",
       x="Mean length (mm)")
```

- Helper checks the same analytic update.
- `rnorm()` simulates 10,000 draws from the *known posterior*, not MCMC.
- `aes()` maps variables; `geom_density()` draws a smoothed density; `labs()` labels the figure.

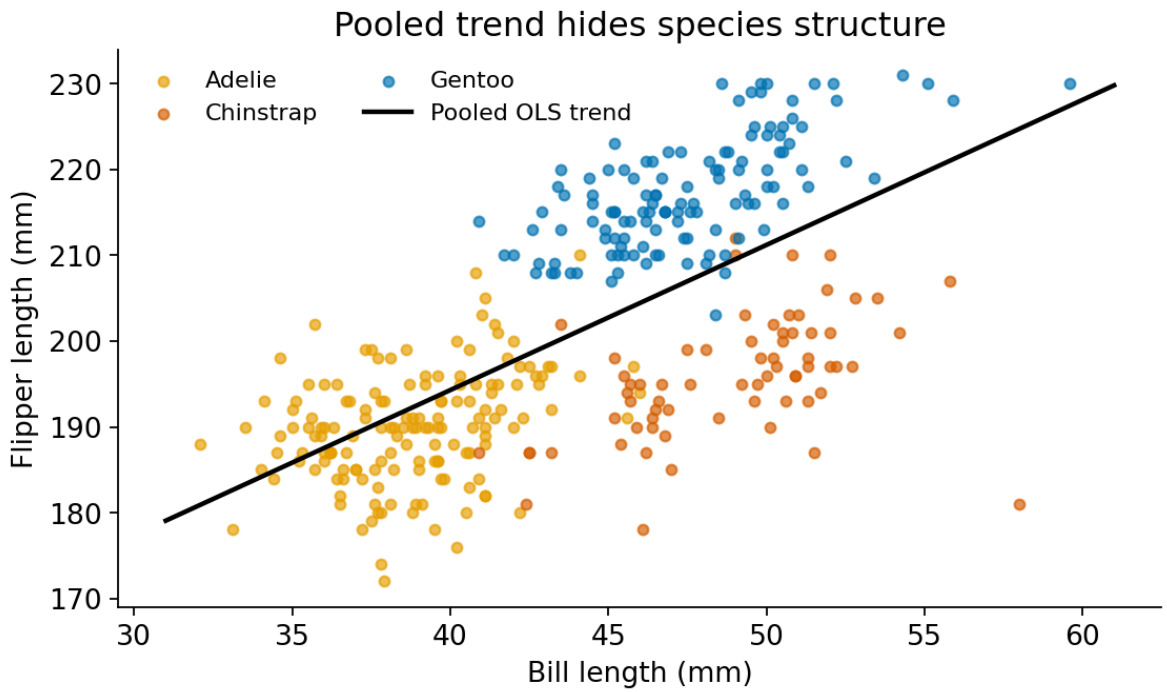
9. Prepare a regression data frame

```
reg9 <- penguins %>%
  # filter(sex == "male") %>%
  select(flipper_length_mm, bill_length_mm) %>%
  drop_na()
head(reg9)
hist(reg9$flipper_length_mm)
```

- `select()` keeps response Y and predictor X.
- `drop_na()` keeps complete cases ($n = 342$).
- The commented `filter()` would instead study males only.
- `head()` previews records; `hist()` checks marginal shape.

10. Plot the bill–flipper relationship

```
ggplot(reg9, aes(bill_length_mm, flipper_length_mm)) +
 geom_point(alpha=.65) +
 geom_smooth(method="lm", se=TRUE) +
 labs(title="Observed bill-flipper relationship")
```



The black line is an *ordinary least squares descriptive trend*, not the Bayesian fitted line. Colors show species; the original code does not color by species.

11. Center the predictor

```
bill_reference <- mean(reg9$bill_length_mm)
reg9 <- reg9 %>%
  mutate(bill_centered = bill_length_mm-bill_reference)
ggplot(reg9, aes(bill_centered, flipper_length_mm)) +
  geom_point(alpha=.65) +
  geom_smooth(method="lm", se=TRUE)
```

$$Y_i = \alpha + \beta(X_i - \bar{X}) + \epsilon_i$$

Centering changes the interpretation of α: it is the expected flipper length *at the sample mean bill length*, not at a biologically unrealistic bill length of zero. Slopes/fitted values are unchanged by centering.

12. Bayesian regression as a generative model

$$Y_i \mid \alpha, \beta, \sigma \sim N(\alpha + \beta x_{ci}, \sigma^2)$$

$$\alpha \sim N(190, 10^2), \quad \beta \sim N(0, 5^2), \quad \sigma \sim \text{Exponential}(0.1)$$

- α is the expected flipper length at the mean bill length.
- β is expected additional flipper mm per additional bill mm.
- σ is residual individual-to-individual spread (positive).
- Exponential rate 0.1 corresponds to prior mean $1/0.1 = 10$ mm.

These priors are illustrative; inspect what they imply before fitting.

13. Fit regression using `rstanarm`

```
fit9_prior_stan <- stan_glm(
 flipper_length_mm ~ bill_centered,
 data=reg9, family=gaussian(),
 prior_intercept=normal(190,10, autoscale=FALSE),
 prior=normal(0,5, autoscale=FALSE),
 prior_aux=exponential(.1, autoscale=FALSE),
 chains=4, iter=10000, seed=84735
)
```

`~` specifies the regression formula; `family=gaussian()` specifies a Normal response; `prior_intercept`, `prior`, and `prior_aux` specify priors on intercept, slope, and residual scale. `autoscale=FALSE` preserves stated scales.

Important: Despite its name, `fit9_prior_stan` is fit to observations and is a *posterior* model.

14. How does Stan get posterior draws?

1. Construct the joint log posterior from likelihood and priors.
2. Run 4 independent Hamiltonian Monte Carlo chains.
3. Use warmup to tune the sampler; keep post-warmup draws.
4. Evaluate mixing, convergence and predictive adequacy.

Code knobs

`chains=4`: multiple diagnostic runs; `iter=10000`: iterations per chain (including warmup, by default); `seed`: reproducible random stream.

MCMC produces approximate samples from a complex posterior; unlike Normal–Normal updating, it usually has no simple closed-form result.

15. Inspect the rstanarm fit

```
prior_summary(fit9_prior_stan)
fit9 <- update(fit9_prior_stan)
summary(fit9)
posterior_interval(fit9, prob=.95)
pp_check(fit9)
plot(fit9, plotfun="trace")
```

- `prior_summary()`: the effective priors.
- `update()` here *refits* the same model, not a special prior-to-posterior step.
- `summary()` gives parameter posterior summaries and MCMC diagnostics.
- `posterior_interval()` gives coefficient/parameter intervals.
- `pp_check()` compares replicates to observations; trace plots examine mixing.

16. Model checking: two separate questions

Computation:
Did MCMC work?

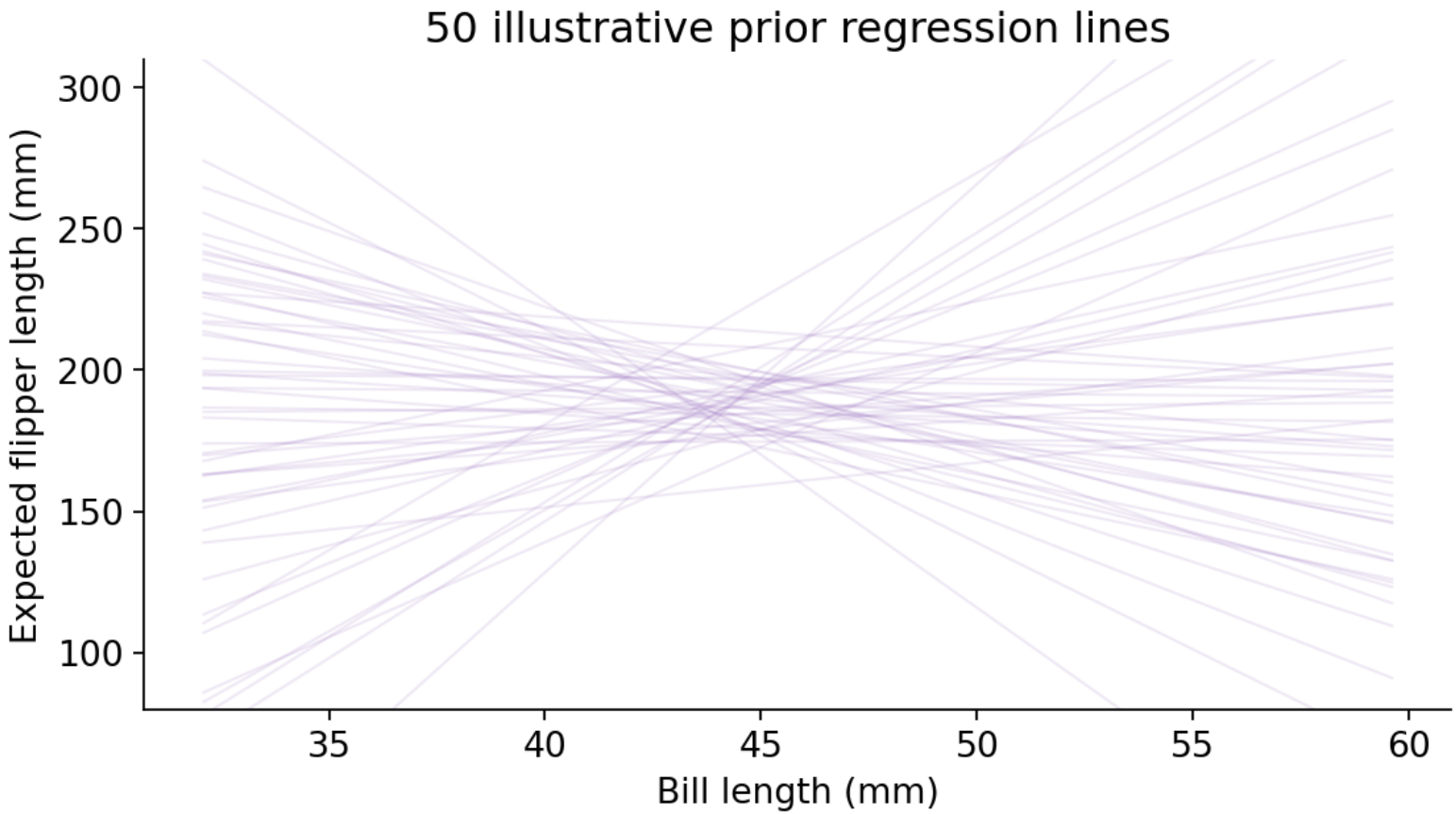
Trace plots should overlap across chains; check $\widehat{R}$ near 1, effective sample size, and sampler warnings/divergences.

Science:
Does model fit?

Posterior predictive checks compare observed patterns with synthetic datasets. Look for systematic structure and overly narrow residual spread.

Excellent MCMC diagnostics do not imply that the statistical model is appropriate.

19. What do prior regression lines look like?



Each line is $\alpha^{(s)} + \beta^{(s)}(x - \bar{x})$, a possible *mean relationship* sampled from the priors. These lines contain **no residual noise**.

20. Extract and subsample prior draws

```
prior_draws <- as.matrix(fit9_prior)
colnames(prior_draws)
set.seed(84735)
prior_draws <- prior_draws[
 sample(seq_len(nrow(prior_draws)),50),,
 drop=FALSE]
x_seq <- seq(min(reg9$bill_centered),
             max(reg9$bill_centered),length.out=100)
```

`as.matrix()` yields rows of draws; `colnames()` verifies parameter names; `sample()` chooses 50 rows; `seq_len()` avoids indexing pitfalls; `drop=FALSE` retains matrix shape; `seq()` defines the line drawing grid.